The system and security model of Troupe 2.0

Aslan Askarov

December 27, 2025

Contents

2025-04-30; AA; this should be a (sub-)section in the paper

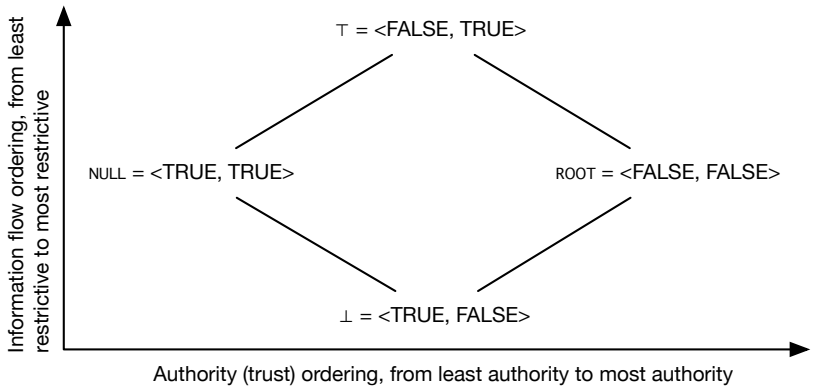

Figure 1: Troupe DC lattice

1 Introduction

This document describes the formal security model of Troupe. The goal of this write-up is two-fold.

1. First, Troupe consolidates several standard techniques from the literature, it is helpful to have a unified presentation of the security model. Along the way, the consolidation leads to minor technical contributions, e.g., specification of the NMIFC constraints for the DC Labels lattice, that deserve proper exposition.
2. Second, we anticipate that some of the material from this document will eventually make it to the draft of the Troupe paper.

2 The label model

We plan to adopt a version of the disjunctive categories labels as follows.

2.1 DC Labels

A security label is a pair $\langle S, I \rangle$, where S and I are positive boolean formulas in *conjunctive normal form*, corresponding to the confidentiality and the integrity components of the label. Figure 1 presents the space of all possible labels. We note the following relevant structures in this space.

2.1.1 The DC Lattice and the SIF and Authority orderings

The SIF ordering The secure information flow ordering is given by identifying the standard information flow lattice structures with the following elements

- Bottom element $\bot = \langle \mathsf{TRUE}, \mathsf{FALSE} \rangle$ corresponding to the most public and the most trusted information.
- Top element $\bot = \langle \mathsf{FALSE}, \mathsf{TRUE} \rangle$ corresponding to the most secret and the least trusted information.
- The flows-to operation $\sqsubseteq$ specifying the restrictiveness of information. Given two levels, $\ell_1 = \langle S_1, I_1 \rangle$ and $\ell_2 = \langle S_2, I_2 \rangle$, we have

$$\frac{S_2 \implies S_1 \qquad I_1 \implies I_2}{\langle S_1, I_1 \rangle \sqsubseteq \langle S_2, I_2 \rangle}$$

- The lattice join $\sqcup$ and meet $\sqcap$ operations, defined as follows

$$\langle S_1, I_1 \rangle \sqcup \langle S_2, I_2 \rangle = \langle S_1 \wedge S_2, I_1 \vee I_2 \rangle$$
$$\langle S_1, I_1 \rangle \sqcap \langle S_2, I_2 \rangle = \langle S_1 \vee S_2, I_1 \wedge I_2 \rangle$$

The Authority ordering The authority (also trust) ordering is given by the following lattice, visually represented in a left-to-right way in Figure 1.

- The null level, $\mathsf{NULL} = \langle \mathsf{TRUE}, \mathsf{TRUE} \rangle$ corresponding to the least trusted security level.
- The root level, $\mathsf{ROOT} = \langle \mathsf{FALSE}, \mathsf{FALSE} \rangle$ corresponding to the most trusted security level.
- The *acts for* relationship $\succeq$ (and a symmetrically defined $\preceq$).

$$\frac{S_1 \implies S_2 \qquad I_1 \implies I_2}{\langle S_1, I_1 \rangle \succeq \langle S_2, I_2 \rangle}$$

- The label coalescing $\diamond$ and dilution $\bowtie$ operator, defined as follows.

$$\langle S_1, I_1 \rangle \diamond \langle S_2, I_2 \rangle = \langle S_1 \wedge S_2, I_1 \wedge I_2 \rangle$$
$$\langle S_1, I_1 \rangle \bowtie \langle S_2, I_2 \rangle = \langle S_1 \vee S_2, I_1 \vee I_2 \rangle$$

Basic properties in the lattice We highlight a few basic algebraic properties that follow directly from the definitions above.

- for all ℓ, $\bot \sqsubseteq \ell$.
- for all ℓ, $\ell \sqsubseteq \top$.
- for all ℓ, $\ell \succeq \mathsf{NULL}$.
- for all ℓ, $\mathsf{ROOT} \succeq \ell$.
- for any ℓ_1, ℓ_2, $\ell_i \succeq \ell_1 \bowtie \ell_2$, i = 1,2
- for any ℓ_1, ℓ_2, $\ell_1 \diamond \ell_2 \succeq \ell_i$, i = 1,2

Placeholder for view and voice operators

Figure 2: View and voice operators

2.2 Downgrading with authority

Definition 1 (Privileged flows-to)**.** *Suppose* $\ell_{auth} = \langle S_{auth}, I_{auth} \rangle$*. Define* privileged flows-to *relation as follows*

$$\frac{S_{auth} \wedge S_2 \implies S_1 \qquad I_{auth} \wedge I_1 \implies I_2}{\langle S_1, I_1 \rangle \sqsubseteq_{\ell_{auth}} \langle S_2, I_2 \rangle}$$

Definition 2 (Rotation operator)**.**

$$\rho(\langle S, I \rangle) = \langle S, \neg I \rangle$$

Proposition 1 (Relationship between privileged flows-to corresponds and standard downgrading)**.**

$$\ell_{from} \sqsubseteq_{\ell_{auth}} \ell_{to} \iff \ell_{from} \sqsubseteq \ell_{to} \sqcup \rho(\ell_{auth})$$

Proof. Immediately by unfolding the definitions. □

2.3 Nonmalleable information flow

We adapt the definitions of view and voice of a label, from the literature on NMFIC, to our setting as follows.

Definition 3 (View of a label)**.** *Define the label* view *operator as follows*

$$\Delta(\langle S, I \rangle) = \langle I, \mathsf{TRUE} \rangle$$

Definition 4 (Voice of a label)**.**

$$\nabla(\langle S, I \rangle) = \langle \mathsf{TRUE}, S \rangle$$

Figure 2 presents the view and voice operators in relationship to the DC label lattice.

Robustness In the NMIFC literature robustness declassification (without authority) is enforced through the following requirements

1. the integrity component of the from and to labels are the same.
2. $\ell_{from} \sqsubseteq \ell_{to} \sqcup \Delta(\ell_{from} \sqcup pc)$, where pc is the pc-label at the time of the declassification.

If we instantiate the second item to the DC label system, we obtain the constraint

$$(I_{from} \vee I_{pc}) \wedge S_{to} \implies S_{from}$$

Putting it together with the privileged flows-to constraint, we have the following top-level rule for declassification with authority

Definition 5 (Robust declassification with authority)**.** *A declassification from level $\ell_{from} = \langle S_{from}, I_{from}\rangle$ to level $\ell_{to} = \langle S_{to}, I_{to}\rangle$ when the program counter is $\langle S_{pc}, I_{pc}\rangle$ is* robust*, if the following two conditions hold*

1. $I_{to} = I_{from}$
2. $(S_{auth} \vee I_{from} \vee I_{pc}) \wedge S_{to} \implies S_{from}$

Transparency For transparency, without authority, the confidentiality level of the information being endorsed limits the endorsement. In our system, it must hold that

$$\langle S_{from}, I_{from}\rangle \sqsubseteq \langle S_{to}, I_{to}\rangle \sqcup \nabla(\langle S_{from}, I_{from}\rangle \sqcup pc)$$

The above then implies the integrity constraint

$$I_{from} \implies I_{to} \vee (S_{from} \wedge S_{pc})$$

Our final definition incorporates authority and is explicit about the constraint that endorsements keep the confidentiality intact.

Definition 6 (Transparent endorsement with authority)**.** *An endorsement from level $\ell_{from} = \langle S_{from}, I_{from}\rangle$ to level $\ell_{to} = \langle S_{to}, I_{to}\rangle$ when the program counter is $\langle S_{pc}, I_{pc}\rangle$ is* transparent*, if the following conditions hold*

1. $S_{to} = S_{from}$
2. $I_{from} \implies I_{to} \vee (S_{from} \wedge S_{pc})$
3. $I_{auth} \wedge I_{from} \implies I_{to}$

Note the apparent surface asymmetry between the definitions is due to rewriting the of the integrity authentication that would otherwise appear in a negative form on the right.

2.4 Example of password checking from the NMIFC paper

2.5 NMIFC limitations

2.6 Overview of downgrading

	Value	Blocking	Mailbox
Confidentiality	declassify	blockdeclto	declmbox
Integrity	endorse	blockdendorseto	endorsembox

3 Authority as capability

3.1 Root-level authority

3.2 Process-level authority

4 Checks and quarantining at cross-boundary

4.1 Basic checks

4.1.1 Outflow mechanisms

When information at level $\ell = \langle S, I \rangle$ is sent to a node n at the trust level $\ell_n = \langle S_n, I_n \rangle$ the runtime checks that

Confidentiality mechanism The sender runtime checks that $S_n \implies S$, which ensures that the receiving node is sufficiently trusted to receive this information. If the check fails, the sending process is halted.

Justification Without this check, the sensitive data leaks to an untrusted node.

Integrity mechanism None. The information is sent with the information flow labels on them.

Justification From the perspective of the sender, no security compromise occurs if a high-integrity information is sent to an untrusted node. Similarly, it is not a problem to send low-integrity information to a trusted node, for as long as it has the right label (but nodes trust their own labeling).

4.1.2 Inflow mechanisms

Consider the scenario when we receive information at level $\ell = \langle S, I \rangle$ from node n, with trust ℓ_n. This information is subject to the following security mechanisms.

Confidentiality mechanism The runtime checks that the $S \implies S_n$, which ensures that the sending node is sufficiently trusted to introduce secrets at level S. If the check fails, the incoming message is quarantined.

Integrity mechanism The runtime raises the integrity level of data to that of $I \vee I_n$. This ensures that the integrity of the node is not compromised.

Overclaiming The mechanisms above prevent *overclaiming* of labels, which is what happens when the label on the information received from n exceeds the trust we have on n.

Overclaiming is a subtle phenomenon in an *open decentralized* system. For integrity, the overclaiming is necessary h

Explain that overclaiming for confidentiality by itself is not a problem for confidentiality but is a problem for availability. Overclaiming for integrity is a problem for integrity

The *overclaiming mechanism discrepancy* Oserve that the treatment of confidentiality and integrity overclaiming is different. For confidentiality, Troupe uses the quarantine mechanism, and for integrity, it raises the taints the level of the messages.

Explain the discrepancy

4.2 Quarantine mechanism

4.2.1 Motivation

Quarantining is a mechanism for step-wise programmable trust leveling, which is typical for authentication and authorization protocols, where the trust between nodes is asymmetric, for example, in a client-to-server authentication scenario, the client trusts the server, but not the other way around.

By default quarantining is disabled. Enabling of the quarantine is a privileged operation requiring (a process-level) authority. Quarantined messages are marked as such by the runtime.

Example showcasing the potential weaknesses without quarantining

2025-05-04; AA / I was thinking that it may be possible achieve what quarantine does by programming, e.g., by introducing a special round-trip to add a temporary authority and raising the trust by juts that, label would be then communicated; so there is generally a non-trivial issue of trust asymmetry.

2025-05-04; AA / Move the authentication example back here

4.2.2 Troupe quarantine protocol

We let *quarantine protocols* to specify what happens to the information that does not pass the inflow check. Currently, Troupe supports only one basic quarantine protocol, but presume that the system may be extended with new protocols, as the need and design arises from future applications. The basic protocol works as follows.

When information from node at trust level ℓ_n enters the system claiming to be at level ℓ, and it is not the case that $\not\ell_n \succeq \ell$, then it is relabeled to $\ell' = \ell_n \bowtie \ell \diamond \ell_q$, where ℓ_q is a fresh *quarantine authority*. The runtime then places the tuple $\ell_q, relabeled - data)$ that consists of the freshly generated authority – to be used for downgrading or endorsement (and/or coalescing with other authorities if needed by program), and the relabeled information. In terms of the Figure 1, this generally pulls the label of the information to the left.

Quarantine authority is added to the metadata record of the message and can be accessed using the metadata pattern of the handler (see. Section 6.1).

4.2.3 Quarantine authority; quarantine capabilities

4.2.4 Quarantine labels and NMIFC

4.2.5 Guarantees of quarantining

- Principle of the safe defaults

Attack	Mitigation
Creating new threads (which affects the scheduler)	Spawn is a capability
Resource exhaustion via creation of new labels	New label is a capability
Divergence	Time boxing
Running out of memory	Time boxing

Table 1: Resource exhaustion attacks

5 Resource exhaustion attacks

2025-04-30; AA; this should be a (sub-)section in the paper

As a design principle, we are careful in Troupe with identifying sources of potential resource exhaustion. Table 1 presents the list of these attacks along with the mitigation strategy.

5.1 Capability-based mitigation of resource exhaustion

For a number of these attacks, Troupe use a capability-based mechanism for mitigating resource exhaustion.

5.1.1 The cost of DC Labels

Maintaining CNF formulas at runtime at a fine-grained level may be expensive, potentially even more so, if we try minimization of representation, either syntactic or semantic. This performance overhead is less of a problem in closed systems where the number of the levels is known ahead of time, or in static systems where the overhead of the checking is paid just once. For a dynamic system like Troupe, extra care needs to be taken to control the overhead.

As a basic optimization, we can introduce a caching layer. Even with that, when the number of levels is unbounded, it a resource exhaustion attack may be possible if the attacker computationally generates new labels continuously. One way to mitigate that can be to make the new-label primitive be a capability, so that untrusted code that does not have that capability cannot introduce new levels into the system.

6 I/O model

6.1 Input handlers

.

A message handler is a single argument function that meets the following conditions

1. The argument passed to the handler function is a tuple $(data, meta)$, where $data$ is value from the mailbox against which the handler is checked, and $meta$ is the message metadata of the value.
2. The handler function must have no side effects, such as send and return
3. The handler must run without errors.

4. It must return a tuple of the form $(hnRet, body)$, where $hnRet$ is 0 to signify that the match is found, and 1 to signify that the match is not found. If the match is found, then the value of $body$ is a single (unit?) argument function corresponding to the body of the handler.

The no-side-effect requirement is important because the handler function may be executed many times over and over until a match is found.

6.2 Message metadata

Message metadata is a record that includes the following information

- The sending node
- The quarantine authority

Other information may be added to the metadata record, to be determined on need.

6.3 Troupe syntactic sugar for handlers

Troupe syntax includes a handler notation of the form

```
hn patVal | patMeta when e1 => e2
```

This notation is desugared to a function as follows.

```
fn input =>
  case input of
      (patVal, patMeta) => if e1 then (0, fn _ => e2)
                           else (1, ())
      _ => (1, ())
```